AskMyPDF — Development Flow

How the project was built, stage by stage

An extractive question-answering system over PDFs, built with Hugging Face `transformers`, Streamlit and EasyOCR.

Overview

The project was built in five stages. Each one ended with something that worked, and each one was tested before the next began. The order matters: the bugs that mattered most were only exposed in Stage 4, when real documents replaced the clean ones we had generated ourselves.

Stage	What was built	How it ended
1	Environment, model selection	Two models confirmed, dependency trap avoided
2	The core pipeline and the interface	A working app on clean documents
3	Test suite and sample documents	18 tests, 7/7 example questions
4	Real documents: papers, scans, 26 languages	Four serious extraction bugs found and fixed
5	Hardening, documentation, delivery	48 tests, docs, pushed to GitHub

---

Stage 1 — Foundations

Goal: choose the models and build an environment that would not fight us later.

Model selection. Two models were checked against the Hugging Face Hub API before a line of code was written — confirming they exist, are tagged `question-answering`, and are actively used:

- `deepset/roberta-base-squad2` — 124M parameters, English, 82.95 F1 on SQuAD 2.0
- `deepset/xlm-roberta-large-squad2` — 560M parameters, ~100 languages

RoBERTa was chosen over plain BERT: identical architecture and size, but pre-trained on roughly ten times more text with a better recipe, so it reads more accurately at the same speed.

The first real problem. The installed `transformers` was version 5, which has **removed** `pipeline("question-answering")` — the entire basis of the project. Rather than rewrite around it, the

dependency was pinned to `transformers>=4.40,<5` and the project given its own virtual environment, so the system-wide installation was left untouched.

Two environment decisions followed from the same instinct:

- `HF_HUB_CACHE` points at the project's own `models/` folder, so downloaded weights stay with the project instead of a hidden user-wide cache.
- `USE_TF=0` stops `transformers` importing TensorFlow — a broken TensorFlow install on the machine was crashing the import.

Stage 2 — The core pipeline

Goal: the shortest path from a PDF to an answer, with each step in its own file.

The pipeline was deliberately split so that each stage could be tested and explained on its own:

```
pdf_extractor.py  →  chunker.py  →  qa_engine.py  →  app.py
   PDF to text      text to chunks   chunks to answer    interface
```

Extraction uses `pdfplumber`, falling back to `PyPDF2` when it fails or finds nothing — two libraries read PDFs differently, and a file that defeats one often yields to the other.

Chunking exists because a transformer reads at most 512 tokens at once. Documents are cut into overlapping windows so that an answer sitting on a boundary still appears complete in at least one chunk.

Inference wraps `pipeline("question-answering")` and asks every chunk the same question, keeping the most confident answer.

The interface was built last and kept free of logic — it only displays what the modules return, which is why the whole pipeline can be tested without a browser.

Sample documents. Rather than depend on a PDF library we did not need at runtime, a small generator writes valid PDFs by hand from the PDF specification — objects, a cross-reference table, byte offsets. Three samples came out of it: a short English page, a three-page English article, and a French page for multilingual testing.

Stage 3 — Testing

Goal: be able to prove the thing works, repeatedly, without clicking.

Three layers of testing were written, all on the standard library's `unittest` so the project gained no test dependency:

1. **Unit tests** — chunk overlap and coverage, page tracking, the extractor fallback, and every error path: blank PDFs, files that are not PDFs, 0-byte uploads, missing files, text too short, empty questions.

2. **End-to-end examples** — real PDFs answered by the real model, each with an expected answer. Three deliberately have **no** answer in the document, because a model that answers everything is worse than one that admits ignorance.
3. **Interface tests** — Streamlit's `AppTest` drives the app headlessly: pick a document, ask a question, switch models, check what appears on screen.

At the end of this stage the app answered 7 of 7 example questions and passed 18 tests. It looked finished. It was not.

Stage 4 — Real documents

Goal: stop testing on documents we made ourselves.

This stage produced every important fix in the project. Our generated PDFs were clean, single-column and plainly worded; real documents are none of those things.

4a. Research papers

The *Attention Is All You Need* and BERT papers were run through the pipeline. Four serious faults surfaced at once:

Symptom	Cause	Fix	Effect
`WeusedtheAdamoptimizer`	pdfplumber's default word spacing is too wide for LaTeX	`x_tolerance=1.5`	2,017 → **6,183** words recovered
Abstract and introduction interleaved	Two-column pages read straight across	Detect the gutter; read one column at a time	"*What does BERT stand for?*" fixed
`self-attention` → `selfattention`	Line-break hyphens removed blindly	Keep the hyphen when its left part appears as a word elsewhere in the document	Real hyphens preserved
Every chunk overflowed the model	400 words assumed to be ~490 tokens	Measure tokens with the model's own tokenizer	900–2,400 token chunks eliminated

The clearest single result: "*What does BERT stand for?*" had been answering **"feature-based approach" with 96% confidence** — confidently wrong, because the abstract and the introduction were being spliced together line by line. After the column fix it answers **"Bidirectional Encoder Representations from Transformers" at 98%**.

4b. A scanned book

A 39-page scanned Telugu story book yielded **10 words** of text in total — a website watermark. Every page was an image. This drove the entire OCR path:

`pypdfium2` renders each page at 300 dpi → EasyOCR reads the characters off the image → the recovered text re-enters the normal pipeline. Roughly 20 seconds per page on a CPU, so the page range is chosen by the user and pages already read are cached.

It also exposed a gap in the app's own logic: those 10 watermark words were just enough to pass the "is there any text?" check, so the app would happily let someone question an effectively empty document. Scanned-PDF detection was added.

4c. Twenty-six languages

A test set was built: one short PDF per language, every passage stating the same facts, so results are comparable. **45 of 55 questions** are answered correctly.

Two findings came out of it:

- **Right-to-left scripts were broken.** Arabic, Urdu and Sindhi were extracting backwards and in "presentation form" characters — `تاج محل` arriving as `ﻞﺤﻣ ﺝﺎﺗ`. A repair step restores reading order; those three languages went from 0/6 to 6/6.
- **Indic conjuncts are lost by the PDF format itself.** In Telugu, Kannada, Bengali and others, a conjunct letter is stored as a single glyph with no mapping back to Unicode, so `ఆర్య` arrives as `ఆ`. No code can recover it from the text layer — but reading the same page as an *image* can. The app gained a "use OCR even if the PDF has text" switch, which takes Telugu, Kannada and Bengali from 1/2 to 2/2.

Stage 5 — Hardening and delivery

Goal: survive a demonstration on a real laptop.

A full review of the codebase against real failure conditions produced a list of defects, each verified before being fixed. The most serious:

- **Scanned PDFs could become unreachable.** If a scan also tripped one of the two parsers, the app raised a fatal error instead of offering OCR — the very feature built for that case. A one-word logic error (`all` where `any` was needed).
- **Memory failures were unreadable.** Low memory surfaced as "*Could not load model with any of the following classes*". It now says the model needs about 2.5 GB free and suggests closing applications or switching models.
- **GPU support with a safety net.** An NVIDIA GPU is used automatically — answering measured 1.7× faster — and both OCR and inference fall back to the CPU if the 4 GB card runs out of memory, rather than crashing.
- **Answers were being thrown away.** When every chunk preferred "no answer", a correct span the model was merely unsure about was discarded. Those are now shown as "closest guesses".

- **Scores were not comparable across chunks.** A raw confidence is normalised inside its own chunk. Ranking now uses the margin between the answer and the chunk's own "no answer" score — the standard SQuAD 2.0 approach.

Delivery: README, a user manual, a one-page architecture diagram, 48 automated tests, and the whole project committed and pushed to GitHub.

What the process taught us

Clean test data hides bugs. Every fault that mattered was invisible until real documents arrived in Stage 4. The generated samples passed from the first day and were still passing while the system was badly broken on anything written in LaTeX.

A confident wrong answer is a plumbing problem. The most alarming failure — 96% confidence on a wrong answer — was not the model misbehaving. It was reading two columns of unrelated text spliced together, and doing its best with what it was given.

Know what you cannot do. The project's limits were measured rather than guessed: about 4 of 10 on dense research papers, no OCR for four Indian scripts, no summarising, no arithmetic. All of it is written down.

Where NLP is used in this project

Every natural-language-processing technique applied, and the file it lives in.

NLP concept	Where	What it does here
Extractive question answering	`qa_engine.py`	Predicts where the answer *starts* and *ends* in the passage and returns that span — it never generates text, so it cannot fabricate a fact
Transfer learning / fine-tuning	`config.py`	Uses RoBERTa and XLM-RoBERTa pre-trained on raw text, then fine-tuned on **SQuAD 2.0** with a span-prediction head
Unanswerable questions, null scoring	`qa_engine.py`	Lets the model decline; the null score also ranks chunks by **(span score − null score)**
Sub-word tokenization	`chunker.py`	BPE / SentencePiece: `photosynthesis` → `photos` + `ynthesis`. Measured ~1.2 tokens/word in English, ~2.7 in Telugu
Context-window limit	`qa_engine.py`	Self-attention reads at most **512 tokens**; `doc_stride` slides a second window when a chunk overflows
Segmentation with overlap	`chunker.py`	Chunks share 50 words so an answer on a boundary stays whole in at least one chunk
Cross-lingual transfer (zero-shot)	multilingual model	Fine-tuned for QA in English only, yet answers in ~100 languages — 25/25 on our Indian-language set
Confidence scoring	`qa_engine.py` → `app.py`	Shows the span's softmax probability, flags low values, and documents that scores are normalised per chunk
Text normalization	`pdf_extractor.py`	De-hyphenation from the document's own vocabulary, bidirectional reordering for Arabic/Urdu, Unicode NFKC
Space-less word segmentation	`chunker.py`	Chinese, Japanese and Thai are chunked by character, since they have no spaces
Script identification	`chunker.py`, `app.py`	Measures non-Latin letters to set chunk size and warn on a model/language mismatch
OCR feeding the pipeline	`ocr.py`	Detection + recognition networks turn a scanned page into text that re-enters the pipeline
Evaluation	`tests/`, `scripts/`	Held-out questions with expected answers, including unanswerable ones; extraction and answering scored separately

Results: 7/7 English examples · 25/25 Indian-language questions on clean text · 45/55 across 26 languages · ~4/10 on dense research papers. The English model's published SQuAD 2.0 score is **82.95 F1 / 79.93 exact match**.

The three ideas the project rests on: a transformer reads only 512 tokens at once, so text must be *chunked with overlap*; a confidence score is comparable only within its own chunk, so chunks are ranked by the *margin over saying nothing*; and a model fine-tuned for QA in English still answers in ~100 languages, because *cross-lingual transfer* carries the skill rather than the language.